

Recursive Programming

Exercise 1: Implement a recursive function *remainder*(*n*, *d*) that computes the remainder of the division of *n* by *d*, using only subtraction and recursive calls.

- Clearly identify the base case(s).
- Clearly state the recursive case.
- Compute the time complexity and space complexity in terms of *nnn* and *ddd*, and justify your answers.

Exercise 2: Implement a recursive function *is_palindrome*(*s*) using Python slicing (e.g., *s*[1:-1]) that returns True if *s* is a palindrome and False otherwise.

- Clearly identify the base case(s).
- Clearly state the recursive rule.
- Compute the time complexity and space complexity (in terms of recursion stack depth) and justify your answers.

Exercise 3: The following algorithm is an optimized version of *is_palindrome*. Instead of using slicing, it uses two indices (left and right) to compare characters from the ends toward the center.

Analyze this implementation and compute its time and space complexity. Explain why it is more efficient than the slicing-based version from Exercise 2.

```
def is_palindrome(s, lt, rt):
    # Base case
    if lt >= rt:
        return True
    # If mismatch, not a palindrome
    if s[lt] != s[rt]:
        return False
    # Recursive call on inner substring (without slicing)
    return is_palindrome(s, lt + 1, rt - 1)

input_s = "Madam".lower()
left = 0
right = len(input_s) - 1
print(is_palindrome(input_s, left, right))
```

Exercise 4: Given the following code that implements the mutually recursive functions *is_even*(*n*) and *is_odd*(*n*):

- Complete the code.
- Write a short call trace for *is_even*(4) and *is_odd*(5).
- Compute the time complexity and space complexity, and justify your answers.

```
def is_even(n):
    if n == 0:
        return _____
    else:
        return is_odd(n - 1)

def is_odd(n):
    if _____:
        return _____
    else:
        return _____(_____)

x = is_even(4) # x is True
```

Exercise 5: The following function generates all possible k-mers of length k from a given list of bases.

```
def generate_kmers(k, list_bases):
    if k == 1:
        return list_bases
    else:
        kmers = []
        prev_kmers = generate_kmers(k - 1, list_bases)
        for mer in prev_kmers:
            for base in list_bases:
                kmers.append(mer + base)
        return kmers

bases=['A', 'C', 'G', 'T']
k = 2
result = generate_kmers(k, bases)
print("K-mers of length ",k, "is: ",result)
```

- Explain how the recursion works. Clearly identify:
 - The base case
 - The recursive case
- How many **k-mers** are generated in terms of:
 - k**
 - b** = list of bases
- Compute the time complexity and justify your answer.
- Compute the space complexity and justify your reasoning.